

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL MENDOZA

Carrera: tecnicatura universitaria en programación

Materia: Programación I

GESTIÓN DE DATOS DE PAÍSES

Integrantes:

Fernando Pimentel

Fabrizio Pimentel

Fecha de entrega: 20/06/2026

INDICE

1. Introducción	1
2. Marco Teórico	1
2.1. Listas	1
2.2. Diccionarios	2
2.3. Funciones	5
2.4. Estructuras Condicionales	6
2.5. Estructuras Repetitivas.....	7
2.6. Ordenamientos	8
2.7. Archivos CSV	9
3. Decisiones técnicas y arquitectura	10
4. Capturas de Ejecución	10
5. Dificultades y Conclusiones	13
6. Bibliografía y Webgrafía	14
7. Enlace video explicativo	14
8. Enlace repositorio GitHub	14

1. INTRODUCCIÓN:

Este es un proyecto que tiene como objetivo desarrollar una aplicación en Python para la gestión de datos de países, permitiéndole al usuario almacenar, consultar, modificar y analizar información de manera organizada mediante un menú de opciones. A través de este trabajo se aplican conceptos fundamentales de programación estudiados durante la cursada, tales como listas, diccionarios, funciones, estructuras condicionales, algoritmos de ordenamiento, estadísticas básicas y manejo de archivos CSV.

Este sistema busca ofrecer una solución sencilla para la administración de información, permitiendo al usuario interactuar mediante un menú de opciones y realizar distintas operaciones sobre los datos almacenados. Asimismo, el uso de archivos CSV posibilita la persistencia de la información entre distintas ejecuciones del programa.

2. MARCO TEORICO:

2.1. LISTAS:

La lista es una estructura de datos incorporada que se utiliza para almacenar una colección ordenada de elementos. Son dinámicos, redimensionables y capaces de almacenar múltiples tipos de datos.

- **Mutable:** los elementos de una lista pueden cambiar, actualizar, agregar, eliminar después de crear la lista.
- **Ordenados:** los elementos mantienen el orden en el que se insertan.
- **Basado en índices:** se accede a los elementos utilizando su posición, comenzando desde el índice 0.

En Python, una lista se declara utilizando corchetes [] y separando los elementos con comas. No es necesario especificar el tamaño ni el tipo de datos que va a contener.

Ejemplo de listas vacías:

```
lista_videojuegos = []  
lista_notas = []  
lista_estudiante = []
```

Ventajas al usar listas:

- **organización más clara:** ya que proporcionan una estructura ordenada que facilita la comprensión y manipulación de los datos.
- **acceso rápido:** permite acceder instantáneamente a cada elemento conociendo su índice.
- **facilita procesos repetitivos:** Simplifica tareas como calcular promedios, ordenar elementos o buscar valores específicos mediante algoritmos eficientes.
- **eficiencia en memoria:** Optimiza el uso de los recursos del sistema al reservar bloques contiguos de memoria para datos relacionados.

2.2. DICCIONARIOS:

Un diccionario es una estructura de datos que permite almacenar información mediante un sistema de pares clave-valor. Es decir, cada elemento que contiene tiene una clave (key) única, asociada a un valor (value). A este tipo de estructura también se lo conoce como tabla de dispersión o mapa asociativo en otros lenguajes de programación. En muchos contextos reales, no basta con guardar datos en forma de lista o conjunto, ya que necesitamos asociar un dato con otro.

- Cada clave es única dentro del diccionario. No puede haber dos claves iguales.
- Los valores pueden ser de cualquier tipo de dato: números, cadenas, listas, otros diccionarios, etc.
- Los diccionarios son mutables: se pueden agregar, modificar o eliminar pares clave-valor después de su creación.
- Son no ordenados hasta versiones anteriores de Python 3.6. A partir de Python 3.7 en adelante, los diccionarios mantienen el orden de inserción, aunque no deben considerarse estructuras ordenadas en el mismo sentido que las listas.

Ejemplo: este ejemplo muestra como un diccionario almacena datos utilizando claves y valores y su salida.

```
data = { "name": "Jake", "age": 22 }  
print(data)
```

```
{'name': 'Jake', 'age': 22}
```

Métodos de los diccionarios en Python:

Los diccionarios en Python no solo almacenan pares clave-valor, sino que también cuentan con un conjunto de métodos incorporados que permiten manipular y consultar su contenido de forma eficiente y sencilla.

Los tres métodos más comunes para este propósito son:

- `keys()`: devuelve un objeto iterable con todas las claves.
- `values()`: devuelve un objeto iterable con todos los valores.
- `items()`: devuelve un objeto iterable con todos los pares clave-valor como tuplas.

Método `keys()`:

El método `keys()` se utiliza para obtener todas las claves que existen dentro de un diccionario.

- No devuelve una lista, sino un objeto de tipo `dict_keys`. Sin embargo, si se necesita una lista, se puede convertir usando `list()`.
- Es útil para recorrer o consultar si cierta clave existe dentro del diccionario.
- Es ideal para usar en estructuras de control como `for`.

Ejemplo:

```
contactos = {  
    "Fernando": 23479310,  
    "Fabricio": 23891043,  
    "Tomas": 23891043,  
}  
claves = contactos.keys()  
lista_claves = list(claves)  
print(lista_claves)  
  
#salida esperada  
["Fernando", "Fabricio", "Tomas"]
```

Método `values()`:

El método `values()` se utiliza para obtener todos los valores almacenados en un diccionario.

- Devuelve todos los valores asociados a las claves del diccionario.

- No devuelve una lista, pero se puede convertir con `list()` si se desea.
- Puede contener valores repetidos, ya que los valores no tienen que ser únicos en un diccionario.
- Es útil para inspeccionar o recorrer los datos almacenados sin importar las claves.

Ejemplo:

```
contactos = {
    "Fernando": 23479310,
    "Fabricio": 23891043,
    "Tomas": 23891043,
}
valores = contactos.values()
lista_valores = list(valores)
print(lista_valores)

#salida esperada
[23479310, 23891043, 23891043]
```

Método `items()`:

El método `items()` permite obtener todos los pares clave-valor del diccionario, agrupados en tuplas.

- Cada elemento del objeto `dict_items` es una tupla con dos elementos: la clave y su valor correspondiente.
- Es muy útil cuando se desea recorrer un diccionario completo, obteniendo tanto la clave como el valor al mismo tiempo.
- Se puede convertir a una lista de tuplas para trabajar más fácilmente si se requiere.

Ejemplo:

```
contactos = {
    "Fernando": 23479310,
    "Fabricio": 23891043,
    "Tomas": 23891043,
}
pares = contactos.items()
lista_pares = list(pares)
print(lista_pares)

#salida esperada
[("Fernando", 23479310), ("Fabricio", 23891043), ("Tomas", 23891043)]
```

2.3. FUNCIONES

Una función es un bloque de código reutilizable que realiza una tarea específica. En lugar de escribir el mismo código varias veces, podemos definir una función y llamarla cuando sea necesario.

Para crear una función, usamos la palabra clave `def`, seguida del nombre de la función y paréntesis `()`.

Parámetros:

Las funciones pueden recibir valores como parámetros. Estos valores permiten que la función realice cálculos o procesos basados en datos externos.

Retorno:

A veces necesitamos que una función devuelva un resultado en lugar de solo imprimirlo. Para esto usamos la palabra clave `return`.

Sintaxis:

En Python, una definición de funciones tiene las siguientes características:

1. Palabra clave `def`
2. Un nombre de función
3. Paréntesis `()`, y dentro de los paréntesis los parámetros de entrada sean opcionales.
4. Dos puntos `:`
5. Algún bloque de código para ejecutar
6. Una sentencia de retorno (opcional)

Beneficios:

1. Reutilización: Podemos llamar la misma función en diferentes partes del código.
2. Modularidad: Se pueden dividir programas complejos en funciones más pequeñas y manejables.
3. Claridad: Hace que el código sea más fácil de entender y mantener.
4. Mantenimiento: Si hay un error en una función, solo se corrige en un solo lugar.
5. Evita redundancia: Se evita escribir el mismo código varias veces.

Ejemplo de función:

```
# función sin parámetros o retorno de valores
def diHola():
    print("Hello!")

diHola() # llamada a la función, 'Hello!' se muestra en la consola

# función con un parámetro
def holaConNombre(name):
    print("Hello " + name + "!")

holaConNombre("Ada") # llamada a la función, 'Hello Ada!' se muestra en la consola

# función con múltiples parámetros con una sentencia de retorno
def multiplica(val1, val2):
    return val1 * val2

multiplica(3, 5) # muestra 15 en la consola
```

2.4. ESTRUCTURAS CONDICIONALES

En Python, los condicionales son estructuras de control que permiten ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

Estructura del condicional

Un condicional en Python debe tener:

Sangrías: es necesario que toda la estructura que va dentro del if tenga sangría ya que Python no se usan llaves.

Doble punto: significa “entonces” y se utiliza luego de la palabra clave.

Palabra clave: if, else, elif.

Palabras clave

if:

Significa “sí”. Es la palabra que abre la estructura condicional y sigue con una instrucción de lo que se debe hacer si se cumple la sentencia comenzada con “if”.

else:

Significa “en case que”. Sigue con la instrucción de lo que se debe hacer si no se cumple la sentencia comenzada con “if”.

elif:

significa “sino, si”. Cumple la misma función que el “if”, pero se utiliza para añadir una segunda condición.

pass:

Se utiliza cuando queremos que el código no haga nada al respecto. Generalmente se usa con else.

Ejemplo de la utilización las estructuras condicionales.

```
# Definimos las variables a y b
a = 10
b = 10

# Si a es menor que b, imprimir "a es menor que b"
if (a < b):
    print("a es menor que b")
# Si a es igual que b, imprimir "a es igual que b"
elif (a == b):
    print("a es igual que b")
# En cualquier otro caso, imprimir "a es menor que b"
else:
    print("b es menor que a")
```

Condicionales anidadas

Los condicionales anidados son condicionales que se encuentran dentro de otros. Se utilizan para evitar para evitar condiciones compuestas y para combinarlos de manera correcta debemos respetar la indentación.

Por ejemplo:

```
if (se_mueve == "si" and deberia_moverse == "si") or (deberia_moverse == "no" and deberia_moverse == "no")
```

Condicionales ternarios

Son condicionales que se escriben en una sola línea, siguiendo el formato:

```
a = 18
b = 20
print("a es menor que b") if (a < b) else print("b es menor que a")
```

a es menor que b

2.5. ESTRUCTURAS REPETITIVAS

En programación, la iteración es el proceso de repetir una serie de instrucciones múltiples veces hasta que se cumpla una condición específica. Esta característica es fundamental porque permite automatizar tareas repetitivas, optimizar el código y reducir la cantidad de líneas necesarias para resolver problemas.

Existen dos tipos principales de iteración:

Ciclos while:

El bucle while en Python permite ejecutar un bloque de código mientras se cumpla una condición lógica. Esto significa que la cantidad de iteraciones no está definida de antemano, sino que depende de la evaluación de una condición booleana. Si la condición del while nunca se vuelve falsa, el bucle se ejecutará indefinidamente, causando que el programa no termine.

Ejemplo, observa el siguiente código con bucle while:

```
contador = 1
while contador <= 5:
    print("Vuelta número", contador)
    contador += 1
print("Fin del programa")
```

Ciclos for:

El bucle for permite repetir una acción un número determinado de veces. A diferencia de while, aquí sabemos de antemano cuántas veces se repetirá el código.

Ejemplo, observa el siguiente código con bucle for:

```
for x in range (5):
    print("Repetición número", x + 1)

print("Fin del programa")
```

Cosas a tener en cuenta:

- **Uso de break:** Si queremos detener el for antes de que termine, usamos break.
- **Uso de continue:** Si queremos saltar una repetición sin detener el ciclo, usamos continue.

2.6. ORDENAMIENTOS

Las listas de Python tienen dos métodos incorporados como: list_sort() que modifica la lista in situ. Y sorted() que crea una nueva lista ordenada a partir de un iterable.

Ambos aceptan parámetros clave para personalizar el orden:

- **Key:** permite especificar una función (como lambda) para definir el criterio de ordenación, útil para ordenar objetos complejos o lista de listas.

- **Reverse:** un valor booleano (por defecto false) que, si se establece en True, invierte el orden para lograr una clasificación descendente.

2.7. ARCHIVOS CSV

Un archivo CSV es un tipo de archivo de texto plano diseñado para almacenar datos en forma de tabla. Aunque al abrirlo con un editor de notas parece simple texto, su estructura interna sigue reglas muy precisas que permiten organizar grandes volúmenes de información de manera eficiente.

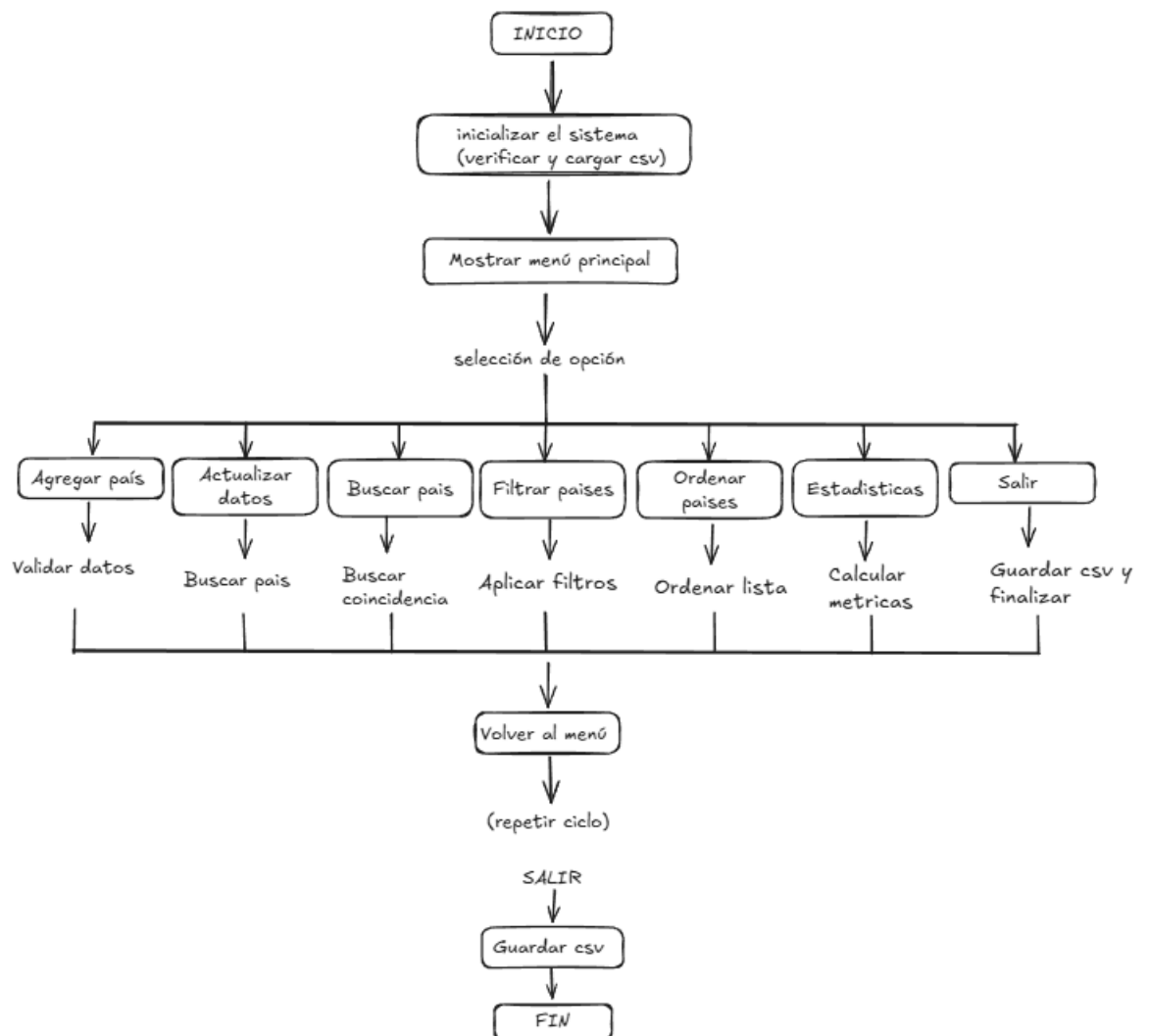
¿Para qué se usa?

En el desarrollo de software y la ciencia de datos, los archivos CSV son fundamentales por varias razones:

- **Interoperabilidad:** Es un lenguaje "universal". Un sistema escrito en Java puede exportar un CSV que luego será procesado por un script en Python sin problemas de compatibilidad.
- **Persistencia:** Permiten guardar los resultados de un programa (datos en RAM) de forma permanente en el almacenamiento del disco.
- **Ligereza:** Al ser texto plano sin formatos complejos (como negritas, colores o fórmulas), ocupan mucho menos espacio que un archivo .xlsx de Excel.
- **Entrada de Datos:** Son ideales para cargar listados masivos, configuraciones o registros históricos en una base de datos.

3. DECISIONES TECNICAS Y ARQUITECTURA

Diagrama de Flujo de operaciones principales



4. EJECUCION DEL PROGRAMA

Menú principal:

```
? Seleccioná una opción: (Use arrow keys)
» Agregar un país
  Actualizar los datos (poblacion, superficie)
  Buscar un país por nombre
  Filtrar países
  Ordenar países
  Mostrar estadísticas
  Salir del programa
```

Descripción: se muestra el menú principal del sistema, donde el usuario selecciona la operación a realizar.

Agregar un país:

```
? Seleccione una opción: Agregar un país
ingrese el nombre del país que desea agregar: uruguay
ingrese la población para Uruguay: 3500000
ingrese la superficie para Uruguay: 176215
ingrese el continente de Uruguay: america
país agregado correctamente
el proceso de registro fue finalizado
```

Descripción: ejemplo de carga de un nuevo país al sistema, cada país deberá cargarse con su población, superficie y continente.

Actualizar los datos:

```
? Seleccione una opción: Actualizar los datos (población, superficie)
===== ACTUALIZAR DATOS =====
ingrese el nombre del país a actualizar sus datos: uruguay
ingrese la nueva población para Uruguay: 3600000
ingrese la nueva superficie de Uruguay: 180000
se actualizó correctamente
```

Descripción: El sistema permite modificar la población y la superficie de un país previamente registrado, actualizando la información almacenada en el archivo CSV.

Buscar un país por nombre:

```
? Seleccione una opción: Buscar un país por nombre
===== BUSCAR PAIS =====
ingrese el nombre del país a buscar: ur

=====
nombre: Uruguay
población: 3600000
superficie: 180000
continente: America
=====
```

Descripción: en la imagen se muestra un ejemplo del resultado de una búsqueda por coincidencia parcial.

Filtrar países:

```
? Seleccione una opción: Filtrar países

1. Continente
2. Rango de población
3. Rango superficie
Ingrese una opción: 1
Continente: america

--- PAISES ENCONTRADOS ---
Nombre: Argentina
Población: 1220
Superficie: 122 km²
Continente: America
-----
Nombre: México
Población: 3000000
Superficie: 2000000 km²
Continente: América
-----
Nombre: Uruguay
Población: 3600000
Superficie: 180000 km²
Continente: America
-----
```

Descripción: en esta opción se realiza un filtrado de los países registrados en el sistema utilizando el continente como criterio de búsqueda, mostrando en pantalla todos los países que pertenecen al continente ingresado.

En este caso se muestran más países por que ya se encontraban cargados al csv.

Ordenar países:

```
? Seleccione una opción: Ordenar países

1. Nombre
2. Población
3. Superficie
Ordenar por: 2
Ingrese el orden (A- ascendente / D- descendente): a
--- PAISES ORDENADOS ---
Nombre: Angola
Población: 1200
Superficie: 1111 km²
Continente: Africa
-----
Nombre: Argentina
Población: 1220
Superficie: 122 km²
Continente: America
-----
Nombre: Senegal
Población: 12000
Superficie: 22000 km²
Continente: Africa
-----
Nombre: Francia
Población: 1000000
Superficie: 233333 km²
Continente: Europa
-----
```

Descripción: en esta opción se realiza el ordenamiento de los países registrados en el sistema utilizando la población como criterio, mostrando los resultados organizados en forma ascendente según la selección del usuario.

Mostrar estadísticas:

```
? Seleccioná una opción: Mostrar estadísticas

ESTADISTICAS
País con mayor poblacion: Argelia
País con menor poblacion: Angola
Promedio poblacion: 1588389361.57
Promedio superficie: 63492411572.86

CANTIDAD DE PAISES POR CONTINENTE
America: 2
Africa: 2
Europa: 1
africa: 1
```

Descripción: Se muestran estadísticas generales de los países registrados, incluyendo el país con mayor y menor población, los promedios de población y superficie, y la cantidad de países por continente.

Salir del programa:

```
? Seleccioná una opción: Salir del programa
Saliendo del programa...
```

Descripción: El sistema finaliza la ejecución del programa y cierra la aplicación.

5. DIFICULTADES Y CONCLUSIONES

Durante el desarrollo del trabajo práctico surgieron algunas dificultades que ayudaron a mejorar la comprensión de los temas vistos.

Una de las principales dificultades fue implementar el control de acentos en los textos. Al principio, el sistema no reconocía como iguales palabras como (México) y (Mexico), lo que generaba problemas en las búsquedas de un nombre o país. Para solucionarlo, se investigó el uso de la librería "unicodedata" y se realizó una función llamada (quitar_acentos), que permite normalizar el texto separando lo ingresado por el usuario en caracteres. De esta forma, se pudieron eliminar los acentos y hacer comparaciones más correctas sin importar cómo el usuario escriba la palabra.

También se mejoró la interacción con el usuario en la consola, agregando la limpieza de pantalla y pausas para que los resultados se puedan leer antes de volver al menú.

En conclusión, este trabajo permitió reforzar conocimientos sobre manejo de archivos, uso de listas y diccionarios, validación de datos y organización del código. Además, se logró hacer un programa más completo y fácil de usar, teniendo en cuenta detalles como la comparación de textos y la experiencia del usuario.

6. BIBLIOGRAFIA Y WEBGRAFIA

<https://www.geeksforgeeks.org/>

<https://www.geeksforgeeks.org/python/python-lists/>

<https://www.geeksforgeeks.org/python/python-dictionary/>

<https://www.freecodecamp.org/espanol/news/guia-de-funciones-de-python-con-ejemplos/>

<https://docs.python.org/es/3.13/howto/sorting.html>

también se utilizó el material de estudio de la carrera.

7. ENLACE VIDEO EXPLICATIVO

<https://youtu.be/L1vOvRK-cl4>

8. ENLACE REPOSITORIO DE GITHUB

https://github.com/fabriciopimentel87-ship-it/TPI_países.git